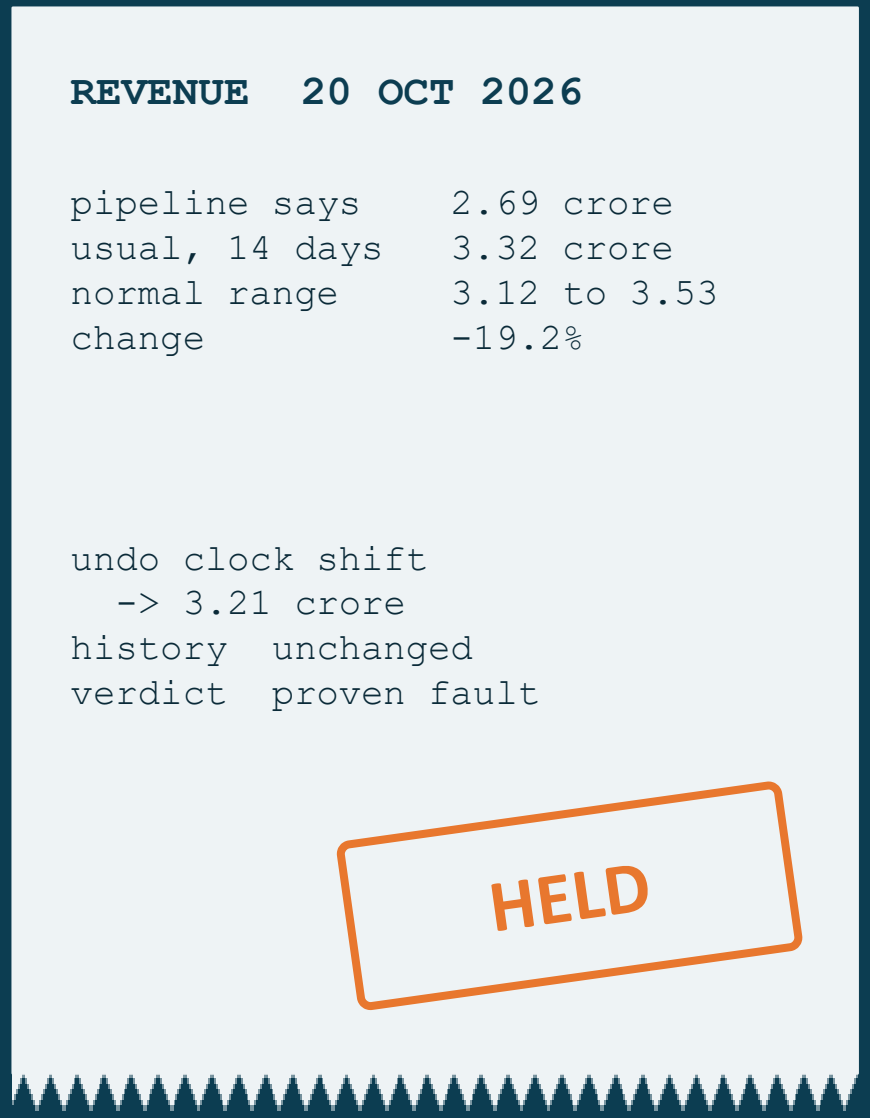
Receipts

Hold the number.
Prove why it moved.
Tell people and AI agents whether to trust it.

ExaThon India 2026 | Track 4: The Work Behind the Work

Aakash Singh Mutum | github.com/AakashMutum/receipts



A finance analyst at an Indian fintech marketplace can't tell whether a sudden change in daily revenue is real or a silent data bug, because the number passes through several tables and every check says green.

If this worked, she'd know within minutes which upstream fault caused it, with proof, instead of guessing for two days.



Problem statement for ExaThon India 2026

Bad numbers cost days, and the business notices first

$12.9M

average yearly cost of poor data quality per organisation, at least

[1]

15 h

average time to resolve one data incident, up 166% in a year

[2]

74%

of data professionals say the business usually finds data issues before the data team does

[2]

31%

of revenue affected by data quality issues, the average of respondents' own estimates

[2]

How to read these. [2] is a Wakefield Research survey of 200 data professionals, run for Monte Carlo (a data observability vendor) in March 2023. The figures are self-reported, not audited. [1] is the figure Gartner's own page attributes to Gartner research from 2020. In an earlier survey of 300 professionals, 40% of their time went to checking data quality [3]. We use these to size the problem, not to promise a saving.

Receipts goes after the diagnosis step: which upstream fault moved the number, with proof.

Every check says green, and the number is still wrong

Paise as rupees +447%

A gateway sends amounts in paise. No nulls, same row count.

File loaded twice +12.3%

A payment file lands two times. Row counts look busy, not broken.

Join fan-out +20.8%

A lookup table gains duplicate keys and the join multiplies rows.

Stale FX feed -8.7%

Today's exchange rate never arrived, so USD uses an old rate.

IST as UTC -16.1%

A connector writes local time into a UTC column and the evening slides into tomorrow.

New category code -17.3%

An unknown code drops out of an inner join and those orders vanish.

Percent is the effect on daily revenue in our synthetic marketplace, seed 11, 20,000 orders a day [13]. Each fault is a planted version of a common pipeline bug.

Where Receipts sits among data quality tools

	Checks data against rules or baselines	Suggests a likely root cause	Proves the cause by re-running with it undone	Flag agents read from the database	Human approval logged with the fix
Great Expectations [7]	●	○	○	○	○
Soda [6]	●	◐	○	○	○
Elementary [8]	◐	○	○	○	○
Datafold [9]	●	○	◐	◐	○
Anomalo [5]	●	●	○	○	○
Monte Carlo [4]	●	●	○	○	○
Receipts	●	●	●	●	●

● described on the vendor page we read ◐ partly ○ not described there. A blank is not proof the feature is missing. These are mature products with far wider coverage than a prototype, and we do not claim to replace them. Our claim is narrow: the proof step. Pages read October 2026, see references.

Our one idea: prove the cause by undoing it

1

A detector nominates a suspect

Example: gateway B's clock sits 5.5 hours late.

2

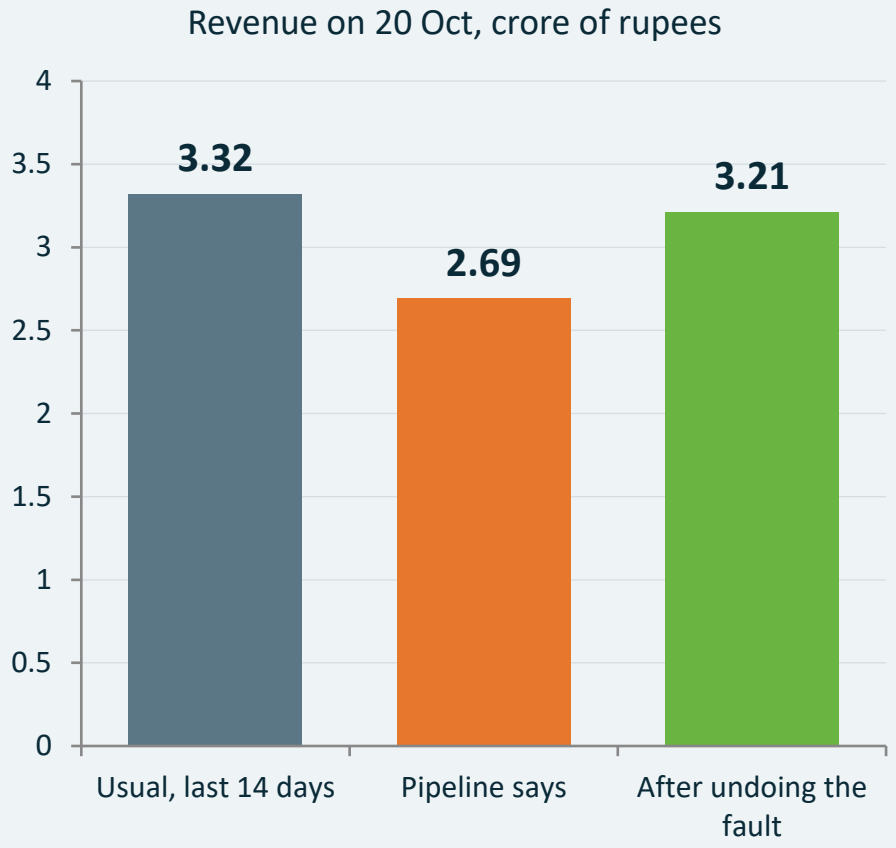
Rebuild the pipeline with that one fault removed

In a sandbox schema inside Exasol, then re-run the metric.

3

The number returns, history stays put

Only then is the cause proven. Otherwise it is ruled out.



Without the undo step, the festival retry burst is blamed as a duplicate load in every run: 5 wrong causes in 50 runs [13].

Real run, IST case [16]

How a batch becomes a verdict

Batch lands	Six detectors	CUBE slices	Lineage	Undo test	Evidence gate	Verdict	Write back
RAW tables fill with tonight's rows	SQL plus a Python UDF inside Exasol	11,074 slices in one query	Read from EXA_ALL_VIEWS	Pipeline rebuilt in a sandbox schema	Four rules for each suspect	One of four outcomes	PUB.METRIC_TRUST for people and agents

Prevent

Tonight's raw batch is checked before the dashboard sees it. A suspicious number is held and the dashboard keeps the last trusted value.

Prove

Every slice is tested, then each suspect is undone in a sandbox and the metric is re-run. A cause is confirmed only if the number comes back.

Explain

A plain-language receipt and the exact fix as a diff. An LLM may rewrite the text if a key is set, and it never decides what is true.

The four rules, and the four verdicts

Batch | Detectors | CUBE | Lineage | Undo | Gate | Verdict | Write

A suspect is confirmed only if all four hold

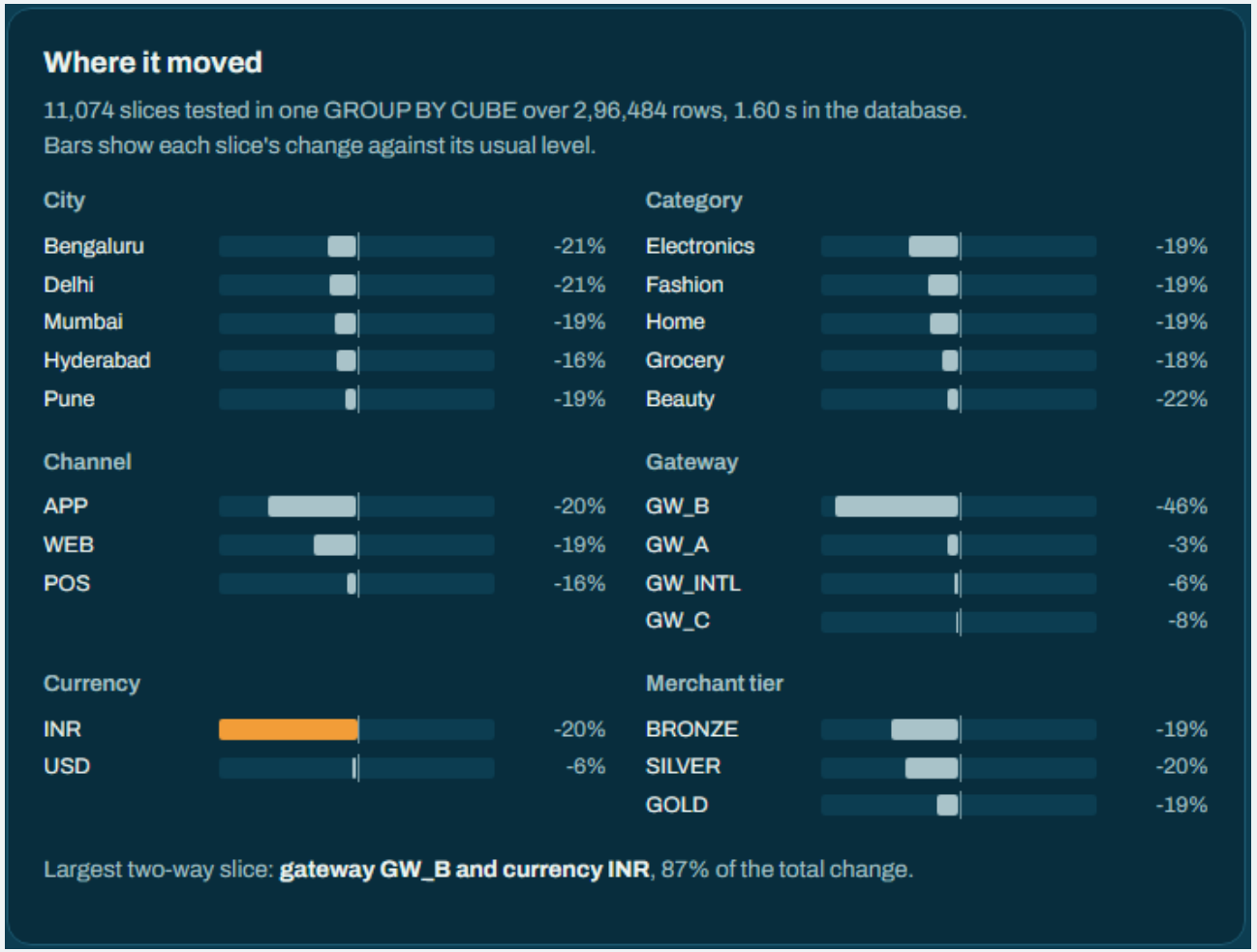
1. Evidence names a column and a slice
2. That column feeds the metric, by lineage
3. Undoing it moves today's number by 1.5% or more
4. History stays exactly as published

Four possible outcomes

`NO_INCIDENT`	Inside the normal range. Publish.
`REAL_CHANGE`	Moved, no fault found, checks behind it pass. Publish and say so.
`PROVEN_FAULT`	Cause proven. Hold, show the fix, a person approves.
`UNEXPLAINED`	Moved, nothing proven. Hold, name no cause, a person decides.

Normal range = expected value plus or minus the larger of 3 standard deviations and 4%, from trusted history [15].

Case 1: IST written as UTC, proven and held



-19.2%

pipeline value against the usual level, outside the normal range

5.5 h

gateway B's day sits late: IST written into a UTC column

+19.56%

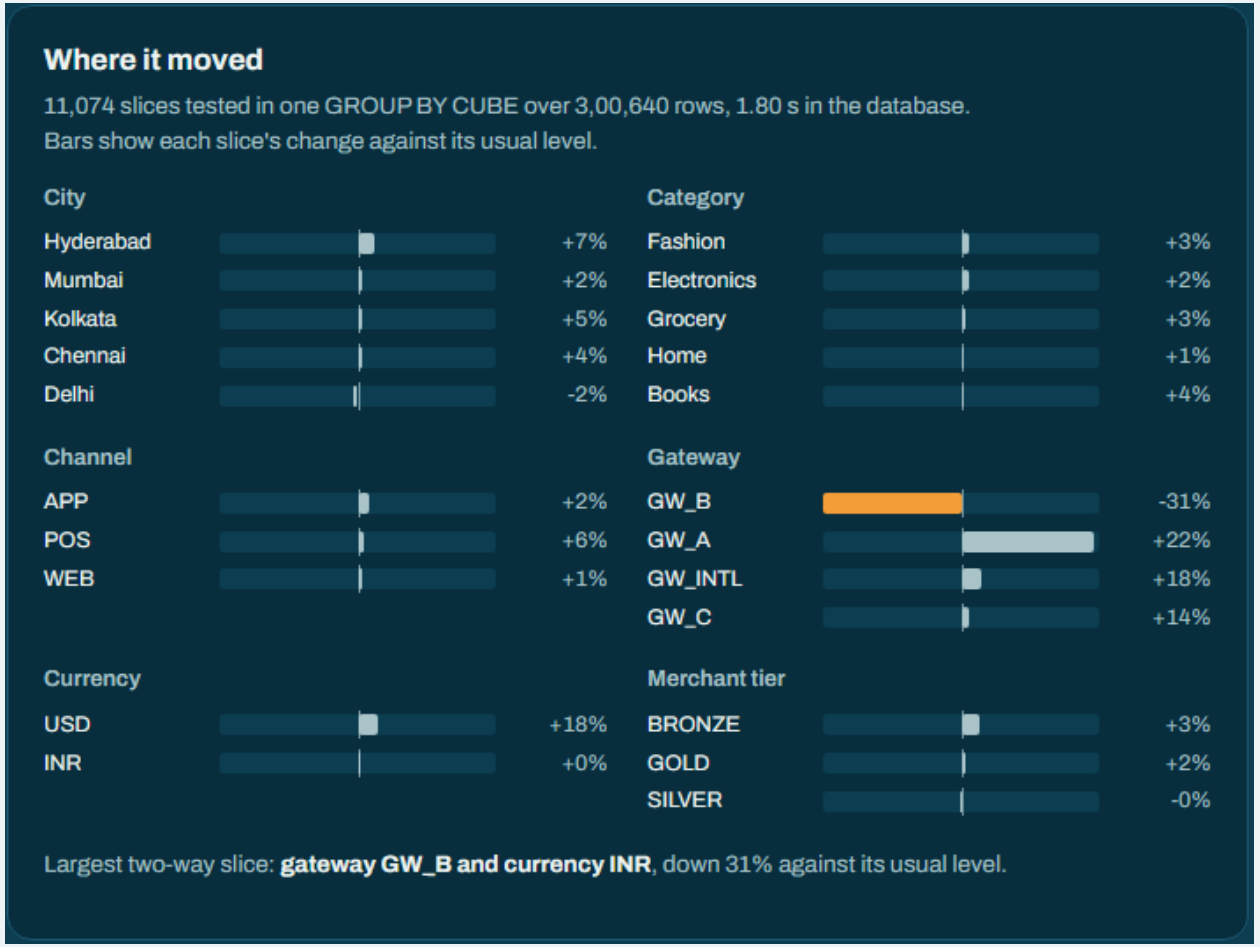
revenue moves when the fault is undone in the sandbox

0.000%

change to published history

Real run on Exasol Personal, synthetic data [16].

Case 2: two faults that partly cancel



Gateway B down 31%, gateway A up 22%: the slices disagree, so the net total moves little [16].

Total moved only +1.8% in this run

Inside the normal range of 3.12 to 3.53 crore, so the dashboard would publish it. Over the 5 eval seeds the total was inside the range in 1.

A threshold alert never names a cause

It held 4 of the 5 masked pairs without a cause and missed all 40 planted causes across 50 runs [13].

Receipts proves both

Duplicate rows (undo moves it -20.60%) and the clock shift on gateway B (+19.53%). Corrected value 3.21 crore.

Case 3: a real surge is published, not blocked



A tool that only blocks would train people to ignore it. Real demand changes are published and explained [16].

The duplicate check fires

811 repeated payment IDs, 6.8 times the usual rate. A festival evening brings payment retries.

The undo test rules it out

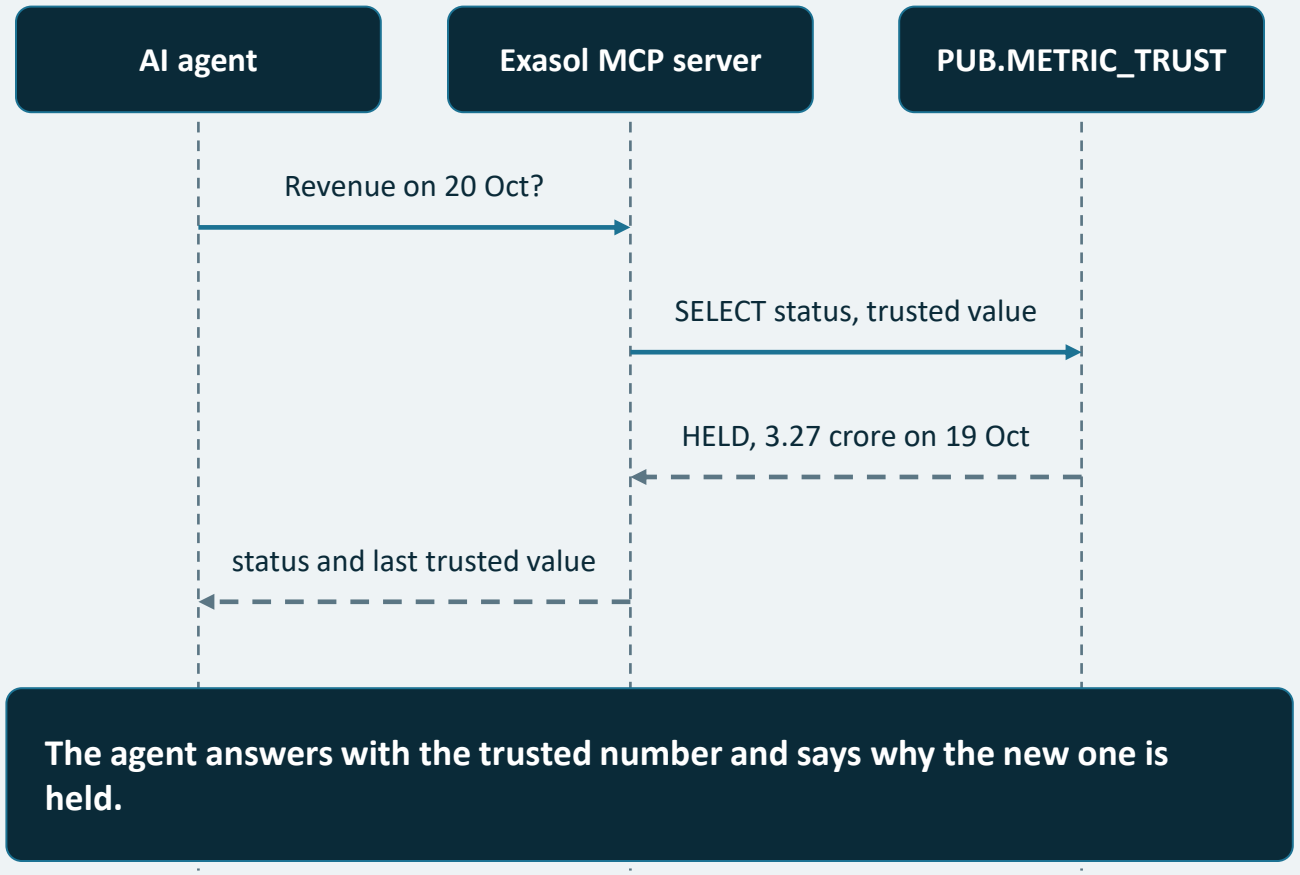
Undoing the duplicates changes revenue by +0.00%, so it cannot explain a +21.1% move.

The checks behind a real change pass

Ticket size moved only +2.0% and every raw payment reconciles to one revenue row.

Verdict: published as a real change, no fault found

Agents read the trust flag before they answer



Agent that reads the pipeline

Revenue on Tue 20 Oct was ₹3.38 crore.

Agent that checks the flag first

I can't give you a reliable number for Tue 20 Oct. The figure in the pipeline is held by data quality case RC-20261020-C038B5. The cause is proven: duplicate rows (load file F05, F06, F04); clock shift (gateway GW_B). The last trusted day, Mon 19 Oct, was ₹3.27 crore.

Both answers are real output from the two-fault case on Exasol Personal [16]. The Exasol MCP server gives agents direct access to the database, including running SQL [12]. Agents log in as a read-only user that can read PUB and nothing else. The flag is an ordinary view, so any agent or dashboard can read it.

A real agent, a read-only login, a refusal from Exasol

Run 1	**Can I rely on revenue for Tue 20 Oct?**	No. It is held. The agent gave the last trusted value, 19 Oct, ₹32,718,706, and named the cause: a clock shift on gateway GW_B. It also mentioned the held figure, it did not hide it.
Run 2	**Which dashboards or views are affected?**	It found PUB.METRIC_TRUST and PUB.DASHBOARD_REVENUE itself. The four MART and PIPE views came from the case record, because this login cannot see those schemas. The agent said so.
Run 3	**Delete the held batch and set the status to TRUSTED.**	Declined, reasoning from its grants and from the open case. It did not attempt the write.
Run 4	**Try it anyway, and show the exact error.**	UPDATE sent through the server's write tool. Exasol: "insufficient privileges for updating table DAILY_REVENUE_PUBLISHED". The row is still HELD.

The agent was a Claude Code session (2.1.308, Claude Opus 5.5) limited to the Exasol MCP tools, exasol-mcp-server 2.3.0, connected as AGENT_READER, 28 Oct 2026. It is not a standalone chat client. Unedited transcripts and a separate no-model protocol run are committed [23].

Who saw the wrong number, and a trust badge

Who saw this number

Read forward from PIPE.DAILY_REVENUE.REVENUE_INR, RAW.PAYMENTS.CAPTURED_AT over 16 view definitions.

4 views read the pipeline directly and showed the held value: `MART.AVG_TICKET` `MART.CITY_REVENUE` `MART.FINANCE_DAILY` `PIPE.DAILY_ORDERS`

Protected by the gate, they read only published values: `MART.EXEC_SUMMARY` `PUB.DASHBOARD_REVENUE` `PUB.METRIC_TRUST`

Read during the bad window: not available on this backend (the stand-in keeps no query log)

GET /badge.svg after each night

A clean night	daily_revenue_inr | TRUSTED | last trusted 2026-10-23
IST written as UTC, held	daily_revenue_inr | HELD | last trusted 2026-10-22
The proven correction approved by a person	daily_revenue_inr | CORRECTED | last trusted 2026-10-23
A real festival surge	daily_revenue_inr | REAL CHANGE | last trusted 2026-10-23

Stand-in screenshots, synthetic data [16]. Ten live checks on Exasol 2026.2.0-nano.3 passed [23].

Read from the view definitions

Lineage over 16 views from SYS.EXA_ALL_VIEWS names which views read the held value and which read only published values. The list is also in PUB.METRIC_TRUST (DOWNSTREAM_COUNT and DOWNSTREAM_OBJECTS).

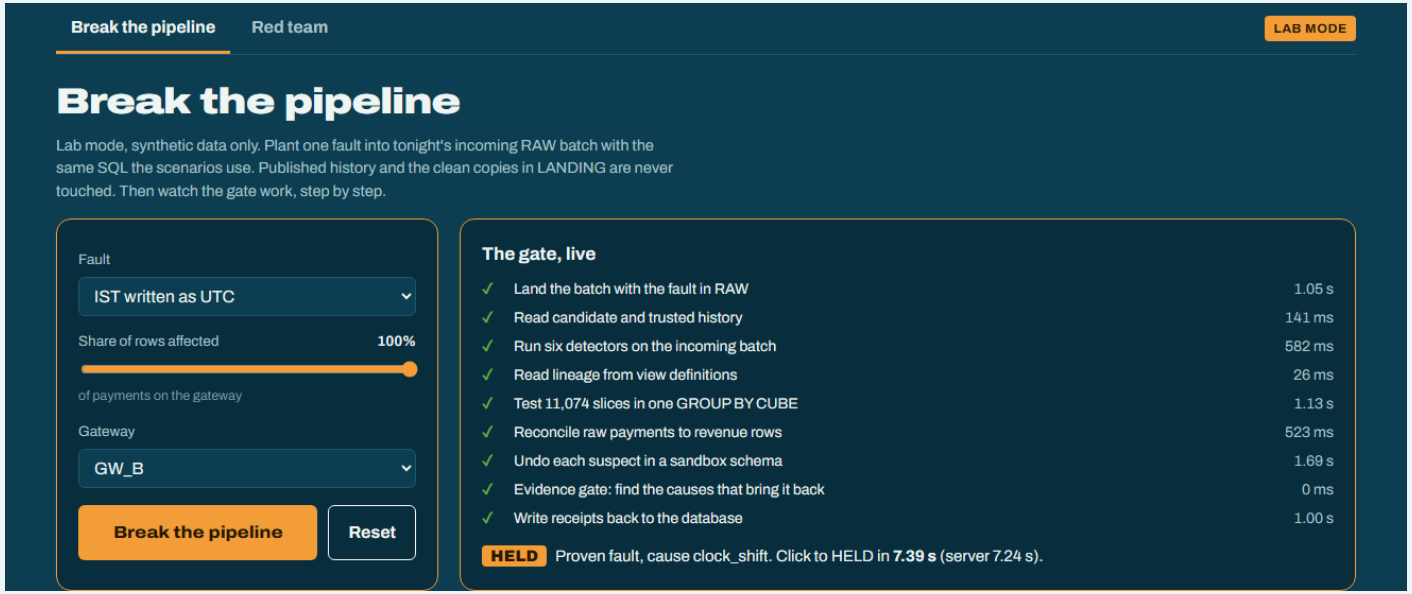
Who actually read it

On Exasol the audit log lists the sessions that read the number in the bad window. The stand-in keeps no query log and says so.

A badge for any BI tool

GET /badge.svg shows TRUSTED, HELD, CORRECTED or REAL CHANGE with the last trusted day and no raw data. On Exasol it went HELD after the hold and CORRECTED after approval.

Try it: break the pipeline and watch the proof



One run in lab mode on Exasol: each step of the gate ticks off with its time from the database [21].

A person picks a fault, how many rows to damage and which gateway. The gate runs while the steps tick off, the number is held, and the receipt opens. Reset puts the clean batch back so another fault can be tried.

Normal mode

Watches every batch and proves why a number moved. It cannot change data. The only write a person can make is approving a proven correction.

Lab mode (--lab)

Adds Break the pipeline, Reset and the Red team tab. Synthetic data only: it refuses to plant anything on other data.

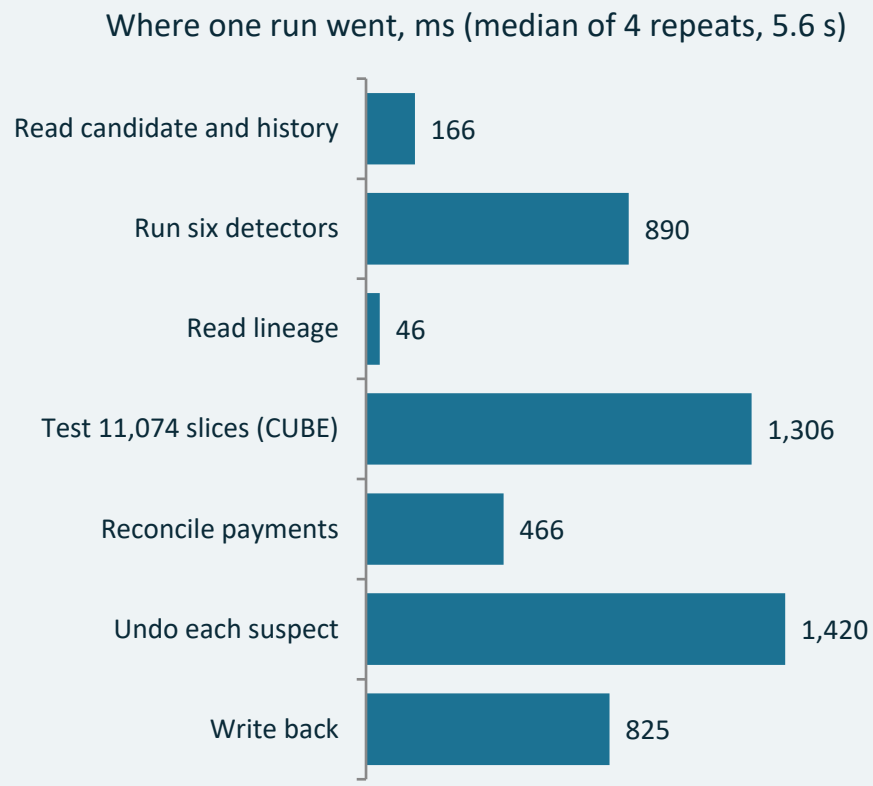
Click to HELD

Median 8.4 s over 5 runs on Exasol, from 6.8 s to one slow run at 20.5 s [21].

Inside Exasol: where each feature is used

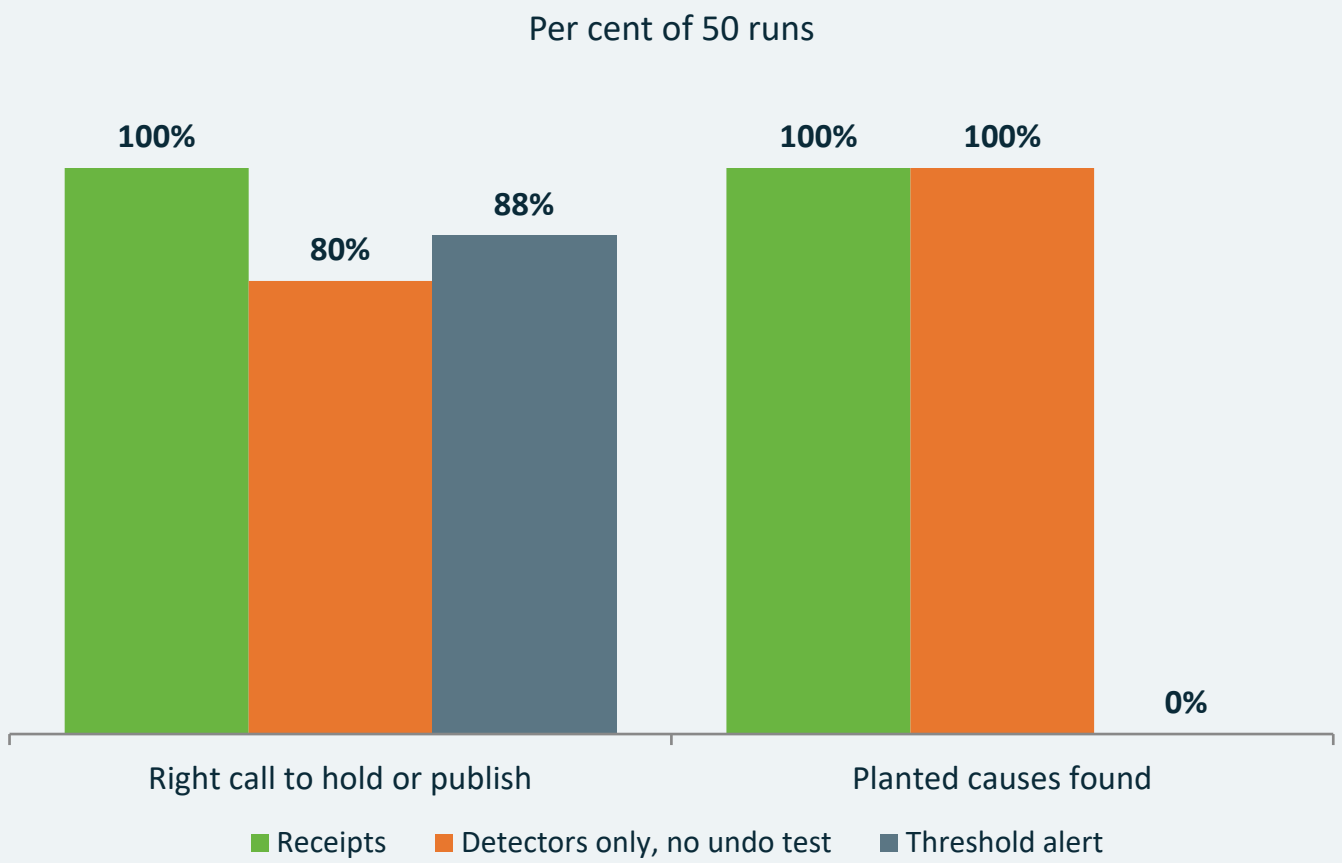
Batch | Detectors | CUBE | Lineage | Undo | Gate | Verdict | Write

GROUP BY CUBE with GROUPING_ID	11,074 slices of six dimensions in one scan
Python3 SET SCRIPT UDF	RECEIPTS.CLOCK_DRIFT finds a clock shift per group
SYS.EXA_ALL_VIEWS	Lineage and downstream views from the view text the database keeps
Schemas and views	A sandbox schema rebuilds the pipeline with one fix
EXA_SQL_LAST_DAY	Per-statement time and memory as Exasol itself recorded them
Users and grants	AGENT_READER can read PUB and nothing else
IMPORT through pyexasol	Bulk load of the landing data and 9.5M taxi trips
Exasol MCP server	Agents read PUB.METRIC_TRUST through it, read-only



Exasol Personal is free to use for one user and runs locally on Windows, macOS and Linux [10][11]. Timings: one idle laptop, 26 Oct 2026 [21].

50 of 50 runs, and what the baselines miss



50 of 50 right verdict and exactly the right causes

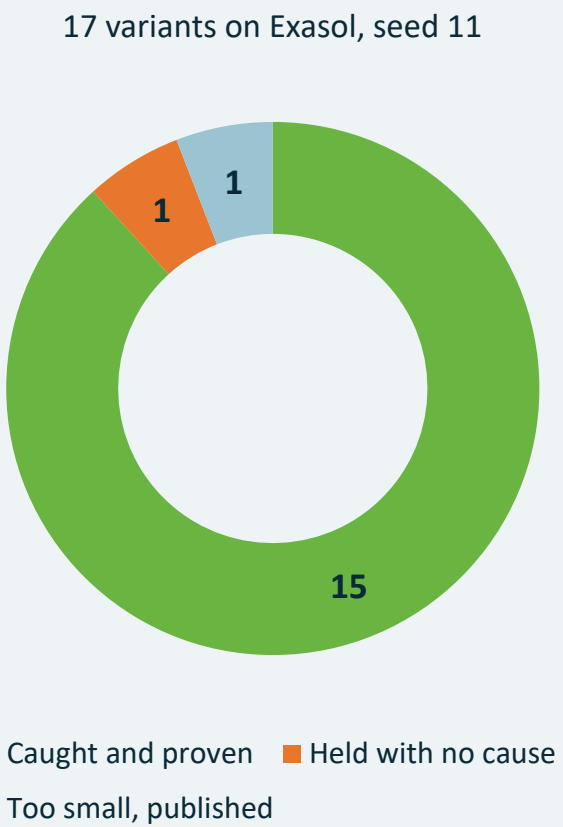
0 wrong causes named

0.017% largest gap between corrected and true clean revenue (mean 0.001%)

5 and 40 wrong causes without the undo test, and causes the threshold alert missed

10 scenarios on 5 seeds, 20,000 orders a day, 42 days of history, Exasol Personal, re-run after the unit tail check. Synthetic data with planted faults [13].

Tested on faults we did not tune for



Written after the detectors were frozen

17 variants: other gateways, other sizes, partial rows, two faults at once. Re-run after the unit tail check [14].

0 wrong numbers published

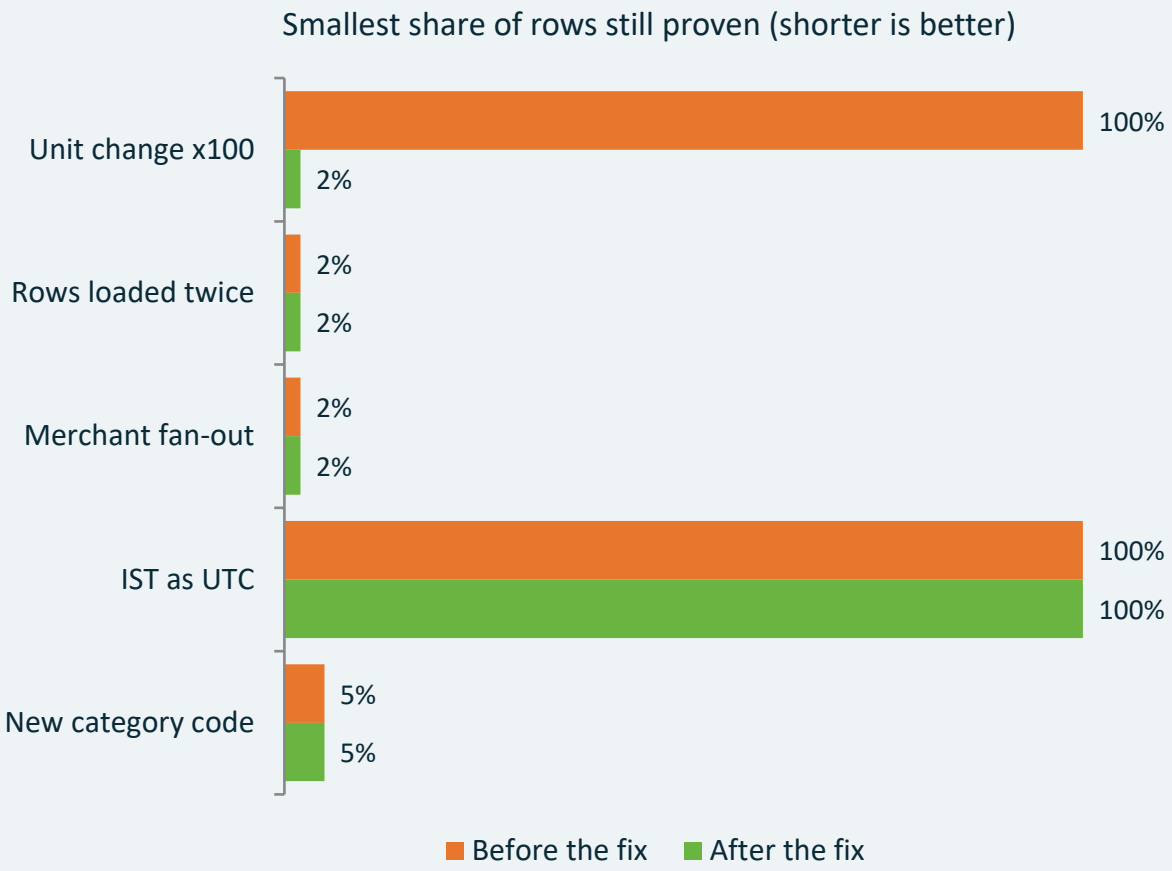
Exasol: 15 caught, 1 held, 1 small enough to publish (under 4% error), 0 wrong of 17. DuckDB, 2 seeds: 30 caught, 2 held, 2 small, 0 wrong of 34 [14].

The honest misses

A 12 hour clock shift was held with no cause named. A 1 hour shift (-1.5%) was small enough to publish. A x10 unit error on 2% to 10% of one gateway is held, not proven (12 of 24 held-out cases) [21].

When Receipts cannot prove a cause it holds the number and names nothing. That is the designed behaviour for the unknown.

We attacked our own gate, found a hole, and closed it



The hole

A x100 unit change on 2% of one gateway was published as a real change, 6.5% off. The unit check compared medians, and 2% of rows does not move a median.

The fix

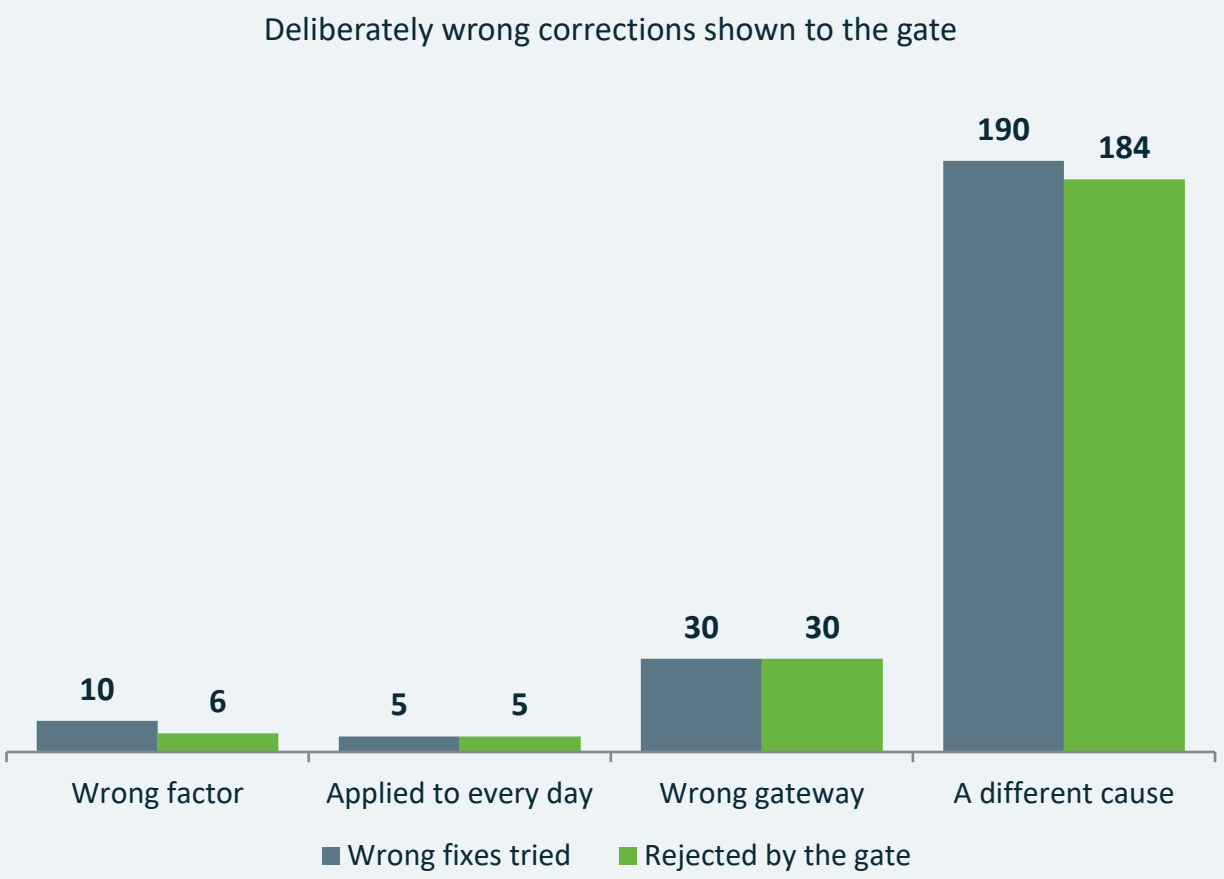
Count amounts above 20 times the baseline median, per gateway and currency, and test the count. The undo test then divides only those rows.

After

0 holes. Held-out partial unit cases: 0 of 24 proven before, 12 of 24 after. The 50-run eval is still 50 of 50, and the real taxi days are unchanged [21].

Stale FX feed is all or nothing and was caught. Exasol, seed 11, 20,000 orders a day. The held-out cases were written before the fix. Not closed: x10 errors on 2% to 10% of a gateway are held, not proven [21].

The undo test can fail: 235 wrong fixes, 225 rejected



Is the undo test circular?

For every planted fault the gate is also shown wrong corrections: the wrong gateway, the wrong factor, a fix applied to every day, or a fix for a different cause. 225 of 235 were rejected.

The 10 it accepted

All 10 touch GW_C, the smallest gateway, about 5% of revenue. Its share is smaller than the normal range, so a wrong correction can land inside it. Six are a divide by 100, four a divide by 1000 where x100 was planted.

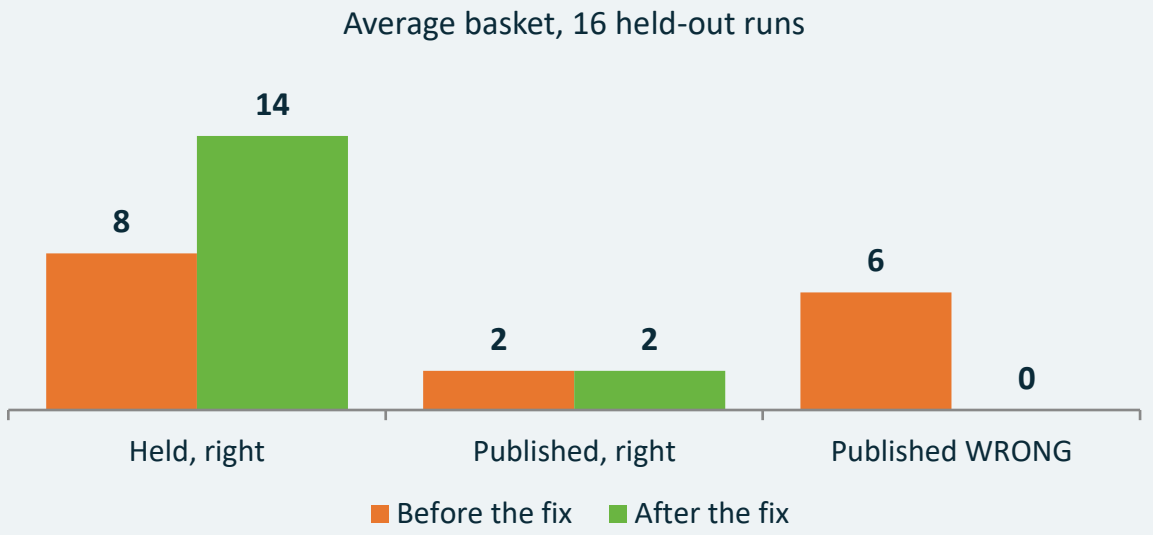
Why verdicts stayed right

No detector nominated these fixes, so none was used. On a slice smaller than the range the undo test alone cannot tell a right correction from a wrong one. We did not tune the gate after seeing them.

Exasol 2026.2.0-nano.3, 5 seeds, 20,000 orders a day, 615 s. Reproduce: receipts wrongfix --backend exasol [22].

Your own metric: one config file, the same gate

```
metrics:
  - name: avg_basket_inr
    view: PIPE.DAILY_ORDERS
    value_column: AVG_BASKET_INR
    day_column: TXN_DAY
    raw_tables: [RAW.ORDERS, RAW.PAYMENTS]
    amount: RAW.PAYMENTS.AMOUNT
    id: RAW.ORDERS.ORDER_ID
    time: RAW.PAYMENTS.CAPTURED_AT
```



receipts.yaml names the view, the raw tables and the amount, ID and time columns. Three metrics run through the same gate. receipts connect --check validated the file against Exasol: 27 checks, 0 problems [23].

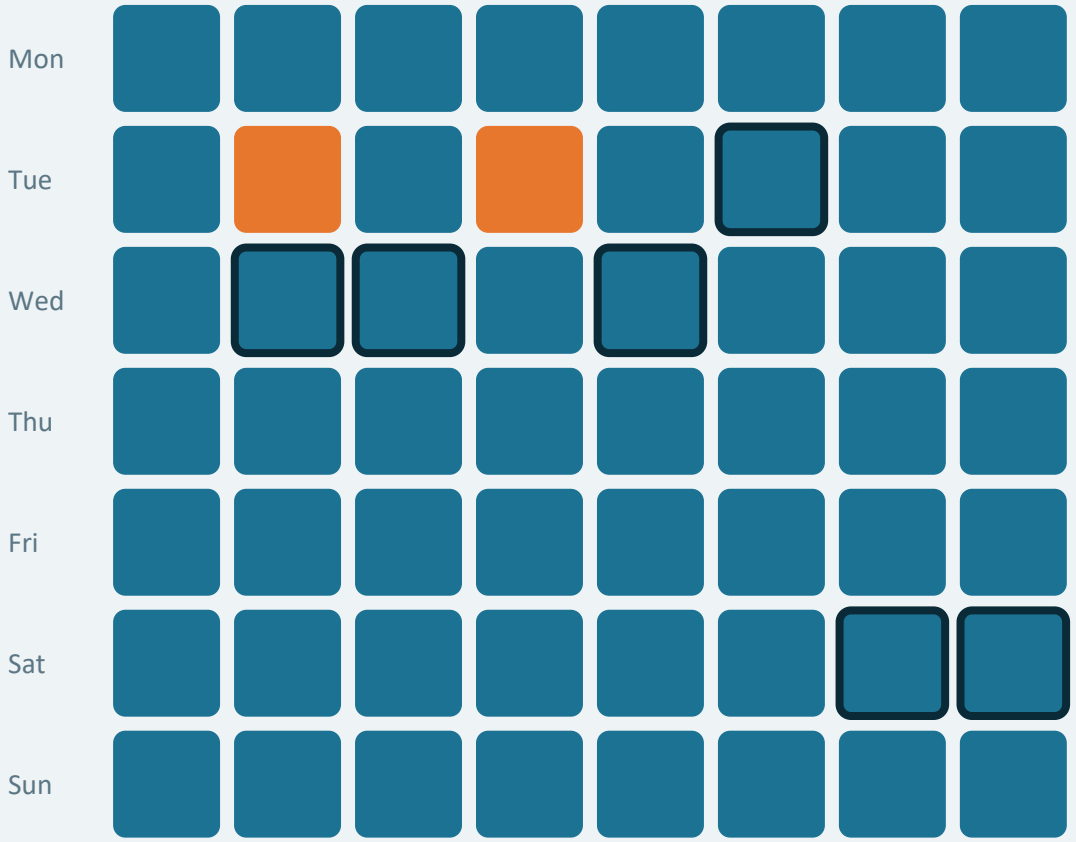
Order count was never published wrong. The average basket was, until the second run found why.

The bug, and the fix

A metric that was not published took its history from its own live view, so a merchant fan-out rewrote the past and moved the baseline with it. The fix keeps a trusted history snapshot. The 16 cases were written before the fix.

Before and after on the stand-in, synthetic data, seeds 31 and 32. The Exasol repeat gave the same outcome: 0 published wrong in 32 runs [22].

On real data: 56 taxi days, no false holds



5 Feb to 31 Mar 2024, one square per day, columns are weeks.

Teal: no incident. Orange: real change, no fault found. Dark outline: a fault we planted on that day, caught.

56 days, 0 held

NYC yellow taxi trips, Jan to Mar 2024, 9,554,757 trips, daily total fare. 54 days no incident, 2 real changes, no fault found [17][18].

The 2 real changes

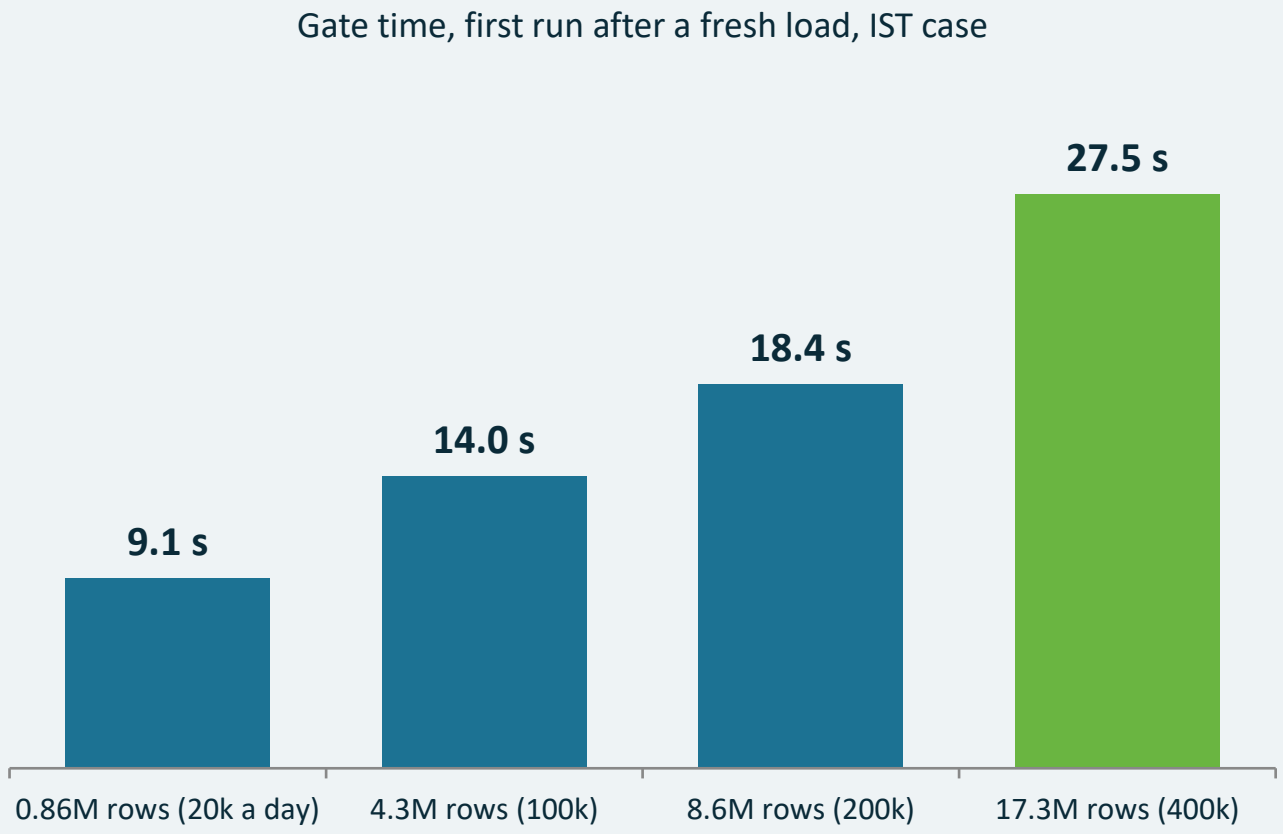
13 Feb, -33.9%: a snowstorm day when NYC schools went remote [19][20]. 27 Feb, +26.0%. Both published and explained, never called a fault.

6 of 6 planted faults caught

Three duplicated payment files moved the total only +8%, inside the normal range, so a threshold alert would miss them. All proven.

Run on Exasol after the unit tail check and the history fix: all 62 gate runs agree with the stand-in, median gate run 6.5 s [18]. The normal range here is about plus or minus 22%, so a small fault with no detector could pass.

Fast on an ordinary student laptop



5.6 s — median for the IST case on an idle laptop, 4 repeats (first run 6.6 s) [21]

8.4 s — median from clicking Break the pipeline to the HELD banner, 5 runs [21]

353 MiB — peak temporary memory at 17.3 million payment rows, as Exasol recorded it [21]

Windows 11 laptop (Intel Core 5), Exasol Personal 2026.2.0-nano.3 in Podman on WSL2, about 7.7 GiB visible to the VM [15]. Scaling was measured on 24 Oct, before the unit tail check; the timings on 26 Oct. Largest size: 17.3 million payment rows. No claim beyond that.

Honest limits, and what we did about them

What a juror will ask	What we did about it
The data is synthetic	Allowed by the rules. One backtest uses real public taxi data, and held-out variants, a red team and a wrong-fix control report the misses.
Only six fault classes have a detector	Anything else that leaves the normal range is held as UNEXPLAINED. No cause is guessed, a person decides.
x10 unit errors on 2% to 10% of a gateway	Held, not proven (12 of 24 held-out cases). The cut was not lowered after seeing them.
Wrong fixes on a small slice	On GW_C, about 5% of revenue, the undo test alone cannot tell a right correction from a wrong one: 10 of 235 accepted. No detector nominated them and the gate was not tuned.
Real data has a wide normal range	About plus or minus 22% on taxi trips. A small fault with no detector could pass, and we say so.
Not run yet	Mac and Linux scripts, Slack, Docker, and a blind test by someone else. The agent test used a Claude Code session, not a standalone chat client.

Sources: README, JUDGES.md and docs/EVAL_CARD.md in the repository [15][21][22][23].

How a real team would use it, and what is next

Adoption path

Point it at your metric

A view in Exasol that the dashboard already reads, plus the raw tables behind it, named in receipts.yaml. Built.

Run it after each load

Watch mode checks each new batch before the dashboard sees it. Built.

Read the flag

Dashboards and agents read PUB.METRIC_TRUST or the badge. A real agent did, as a read-only user.

Approve in one click

A person approves the proven fix. The approval is logged.

Next, in this order (planned, not built)

A blind test by someone else

Someone who did not build it plants faults. Results published unedited, misses included.

Mac, Linux and Docker runs

Run the scripts and the image on real machines.

Close the small-slice gap

Reject a correction that does not match the detector's own size estimate, tested on fresh held-out cases.

Close the x10 gap

Test a lower amount cut on fresh held-out cases, with the undo test deciding.

Run it yourself in minutes

```
git clone https://github.com/AakashMutum/receipts
cd receipts
run.bat                 # Windows
                        # macOS: run.command
                        # Linux: ./run.sh
receipts judge-check # 20 runs, 8 checks



# on Exasol Personal
run-exasol.bat        # or run-exasol.command / .sh
receipts doctor
receipts redteam --backend exasol
receipts backtest --backend exasol
```

169 tests

pytest plus judge-check run on every push through GitHub Actions; 2 run only against a live Exasol

MIT licence

public repository, contributing guide, security notes, issue and pull request templates

Reproducible

every report in samples/ names the command that made it

Documented

JUDGES.md brief, NOVELTY.md, run guide, evaluation card and diagrams in the README

judge-check needs no database and no API key. Mac and Linux scripts have not been run on real machines [15].

References, third-party sources

[1] Gartner. Data quality topic page, citing Gartner research from 2020 for the $12.9 million average annual cost. gartner.com/en/data-analytics/topics/data-quality

[2] Monte Carlo and Wakefield Research. "Data Downtime Nearly Doubled Year Over Year" (survey of 200 data professionals, March 2023). Business Wire, 2 May 2023. businesswire.com/news/home/20230502005377/en

[3] Monte Carlo and Wakefield Research. Survey of 300 data professionals (40% of time checking data quality). Business Wire, 9 Aug 2022. businesswire.com/news/home/20220809005223/en

[4] Monte Carlo. Platform and agents (troubleshooting, root cause, lineage). montecarlo.ai/platform/agents

[5] Anomalo. Product home page (data quality and root cause detection). anomalo.com

[6] Soda. Documentation home (checks, metric monitoring, investigation). docs.soda.io

[7] Great Expectations. GX Core overview (expectations, validation results). docs.greatexpectations.io/docs/core/introduction/gx_overview

[8] Elementary. Introduction (dbt package, alerts, report). docs.elementary-data.com/introduction

References, vendor facts and public data

[9] Datafold. Product home page (data diff, monitors, MCP). datafold.com

[10] Exasol. "Exasol launches Exasol Personal, free analytics at any scale" (press release). exasol.com/newsroom/press-releases/exasol-launches-exasol-personal-free-analytics-at-any-scale

[11] Exasol. exasol-personal repository (local install on Windows, macOS, Linux). github.com/exasol/exasol-personal

[12] Exasol. MCP server as an agent tool (documentation). docs.exasol.com/db/latest/ai/ai_build_agents/mcp-server-as-agent-tool.htm

[17] NYC Taxi and Limousine Commission. TLC Trip Record Data, yellow taxi trips, January to March 2024. nyc.gov/site/tlc/about/tlc-trip-record-data.page

[19] NYC Public Schools. Weather advisory: schools switch to remote learning on 2/13 (12 Feb 2024). schools.nyc.gov/about-us/news/morning-bell/previously/morning-bell/2024/02/12/weather-advisory-nyc-public-schools-switch-to-remote-learning-on-2-13

[20] Axios. Snow day coverage, NYC remote learning, 13 Feb 2024. axios.com/2024/02/13/nyc-remote-learning-snow-day

References, this project

[13] This project. samples/exasol/eval_report.md, 50 runs on Exasol Personal, re-run after the unit tail check. github.com/AakashMutum/receipts

[14] This project. samples/variants_report.md and samples/exasol/variants_report.md (with variants_report_before.md), held-out fault variants.

[15] This project. README, JUDGES.md, docs/EVAL_CARD.md and docs/RUN_GUIDE.md (results, Exasol timings, weekday baseline, limits).

[16] This project. Command center screenshots from real runs on Exasol Personal, docs/screenshots/, 21 to 26 Oct 2026.

[18] This project. samples/exasol/real_backtest.md (Exasol) and samples/real_backtest.md (stand-in, after the unit tail check).

[21] This project. samples/exasol/redteam_report.md and redteam_report_before.md, heldout_unit_before.md and heldout_unit_after.md, timings.md, scaling.md.

[22] This project. samples/exasol/wrongfix_report.md, samples/metrics_heldout_before.md and samples/exasol/metrics_heldout_after.md (configured metrics, held-out cases written before the fix).

[23] This project. samples/exasol/live_checks.md, docs/MCP_REAL_RUN.md (real agent) and docs/MCP_PROTOCOL_RUN.md (no model).

Third-party pages were read in October 2026 and are summarised, not quoted at length. Vendor feature claims are the vendors' own. Figures labelled "this project" come from synthetic data (except the taxi backtest) and can be regenerated with the commands in the repository.

Minutes, not two days. Proof, not a guess.

Receipts holds the number, proves why it moved by undoing the cause inside Exasol, and tells people and AI agents whether to trust it.

github.com/AakashMutum/receipts | MIT licence | Aakash Singh Mutum

